StockFlow

Backend Engineering Internship

Case Study Submission

Role	Backend Engineering Intern
Platform	StockFlow — B2B Inventory SaaS
Date	April 8, 2026
Time Taken	~85 minutes (within 90-min limit)

Part 1 — Code Review & Debugging

1.1 Issues Identified

Issue 1 — No Input Validation

The code accesses request fields (data['name'], data['sku'], etc.) without first validating that they exist or have the expected type. If any required key is absent or the request body is not valid JSON, Python raises a KeyError or TypeError that Flask surfaces as an unhandled 500 error, exposing an internal stack trace.

- Production Impact: Malformed requests crash the endpoint. Clients receive no actionable error message, and internal details may be leaked in the response body.

Issue 2 — No SKU Uniqueness Check

The code inserts the provided SKU with no prior uniqueness check. It relies implicitly on a database-level UNIQUE constraint, which may not exist. Even where it does exist, the resulting IntegrityError is left unhandled.

- Production Impact: Without an application-level guard, duplicate SKUs can corrupt inventory tracking and break any downstream logic that assumes SKUs are unique. Without a DB constraint, silent duplicates are created.

Issue 3 — Split Transactions (Broken Atomicity)

There are two separate db.session.commit() calls — one for the Product and one for the Inventory record. If the second commit fails after the first has already been committed, the product row is persisted with no corresponding inventory entry.

- Production Impact: Orphaned products appear in the system without any stock record, causing phantom zero-stock listings and requiring manual database cleanup.

Issue 4 — Monetary Value Stored as Float

The price field is read from JSON and passed directly into the model as a Python float. IEEE 754 binary floating-point cannot represent many decimal values exactly.

- Production Impact: A price of 19.99 may be stored as 19.989999999..., leading to subtle errors in billing calculations, financial reporting, and tax computation.

Issue 5 — No Authentication or Authorization

The endpoint has no authentication check. Any caller — authenticated or not — can create products. There is also no authorization check to verify that the requesting user has permission to add products to the specified warehouse.

- Production Impact: Unauthenticated writes are possible. In a multi-tenant SaaS product, this could allow cross-tenant data injection.

Issue 6 — warehouse_id Stored on the Product Model (Incorrect Design)

The original code stores warehouse_id directly on the Product record. In a system where a single product can exist across multiple warehouses, this is architecturally wrong. The warehouse-product relationship belongs in the Inventory table, which is the join between products and warehouses.

- Production Impact: The schema cannot represent a product in more than one warehouse without creating duplicate product rows, violating the single-source-of-truth principle for product data.

Issue 7 — Incorrect or Missing HTTP Status Codes

The success path returns a plain dict with no explicit status code. Flask defaults this to 200 OK, but the semantically correct code for a successful resource creation is 201 Created. More critically, no error paths are coded at all, so validation failures, conflicts, and server errors would all fall through to generic unhandled exceptions.

- Production Impact: Clients cannot distinguish success from failure by status code alone. API consumers expecting 201 on creation will break.

Issue 8 — No Error Handling

There is no try/except block anywhere in the function. Any database error, constraint violation, or unexpected condition will produce an unhandled exception. Flask will return a 500 response containing the full Python stack trace.

- Production Impact: Stack traces expose internal implementation details (file paths, ORM queries, table names) to the caller, which is both a security risk and a poor user experience.

Issue 9 — Optional Fields Not Handled

The case study notes that some fields are optional, but the code uses direct dictionary access (data['field']) for every field. Any missing optional field raises a KeyError rather than falling back to a default value.

- Production Impact: Clients cannot omit optional fields. Every optional field becomes effectively required, reducing API flexibility and usability.

1.2 Corrected Production-Ready Implementation

Design Note: In the corrected version, warehouse_id is removed from the Product model. Products are owned by a company; their presence in a warehouse is tracked exclusively through the Inventory table. A company_id field is added to the Product model to enforce multi-tenancy. Price is stored as Python Decimal and mapped to NUMERIC(12,2) in the database. SKUs are treated as unique per company (not globally), consistent with the schema design in Part 2.

```
# ────────────────────────────────────────────────────────────────────
# StockFlow — Fixed create_product endpoint
# ────────────────────────────────────────────────────────────────────
from flask import Blueprint, request, jsonify, g
from decimal import Decimal, InvalidOperation
from sqlalchemy.exc import IntegrityError
from models import db, Product, Inventory, Warehouse
from auth import login_required  # Project-level auth decorator

products_bp = Blueprint('products', __name__)


@products_bp.route('/api/products', methods=['POST'])
@login_required  # FIX 5: Authenticate every mutating endpoint
def create_product():
    """
    Create a new product and register its initial inventory
    in the specified warehouse.

    Expected JSON body:
    {
        "name":             str  (required)
        "sku":              str  (required, unique per company)
        "price":            str  (required, stored as NUMERIC)
        "warehouse_id":     int  (required)
        "initial_quantity": int  (optional, default 0)
        "description":      str  (optional)
    }
    """
```

```
    # FIX 1: Validate that the request body is parseable JSON
    data = request.get_json(silent=True)
    if not data:
        return jsonify({'error': 'Request body must be valid JSON'}), 400

    # FIX 1 (cont.): Validate required fields before touching the DB
    required_fields = ['name', 'sku', 'price', 'warehouse_id']
    missing = [f for f in required_fields if f not in data]
    if missing:
        return jsonify({
            'error': f'Missing required fields: {", ".join(missing)}'
        }), 400

    # FIX 4: Parse price as Decimal to avoid float precision errors.
    # str() ensures that a JSON number like 19.99 does not first pass
    # through float, which would introduce rounding errors.
    try:
        price = Decimal(str(data['price'])).quantize(Decimal('0.01'))
        if price < 0:
            raise ValueError('Price cannot be negative')
    except (InvalidOperation, ValueError) as e:
        return jsonify({'error': f'Invalid price: {e}'}), 400

    # Validate warehouse_id is a valid integer
    try:
        warehouse_id = int(data['warehouse_id'])
    except (TypeError, ValueError):
        return jsonify({'error': 'warehouse_id must be an integer'}), 400

    # FIX 9: Handle optional fields with safe defaults
    description = data.get('description', None)
    try:
        initial_quantity = int(data.get('initial_quantity', 0))
        if initial_quantity < 0:
            raise ValueError()
    except (TypeError, ValueError):
        return jsonify({
            'error': 'initial_quantity must be a non-negative integer'
        }), 400

    # FIX 2: Check for SKU uniqueness within this company before
    # attempting an insert. SKUs are scoped per company, not globally.
    # The DB UNIQUE(company_id, sku) constraint is a safety net for
    # concurrent requests; this check gives a clear error message.
    if Product.query.filter_by(
        company_id=g.current_user.company_id,
        sku=data['sku']
    ).first():
        return jsonify({
            'error': f'SKU "{data["sku"]}" already exists for this company'
        }), 409

    # FIX 5 (cont.): Authorization — verify the warehouse belongs to
    # the authenticated user's company before allowing writes.
    warehouse = Warehouse.query.filter_by(
        id=warehouse_id,
        company_id=g.current_user.company_id
    ).first()
    if not warehouse:
        return jsonify({
            'error': 'Warehouse not found or access denied'
```

```
        }), 404

    # FIX 3: Use a single atomic transaction.
    # db.session.flush() assigns product.id without committing,
    # so we can reference it in the Inventory record. Both rows
    # are committed together, or both are rolled back on failure.
    #
    # FIX 6: warehouse_id is NOT stored on Product. Product belongs
    # to a company; its warehouse presence is recorded in Inventory.
    try:
        product = Product(
            name=data['name'],
            sku=data['sku'],
            price=price,          # FIX 4: Decimal, not float
            description=description,
            company_id=g.current_user.company_id
        )
        db.session.add(product)
        db.session.flush()        # Obtain product.id without committing

        inventory = Inventory(
            product_id=product.id,
            warehouse_id=warehouse_id,
            quantity=initial_quantity
        )
        db.session.add(inventory)

        db.session.commit()       # FIX 3: Single atomic commit

    except IntegrityError:
        db.session.rollback()
        # Handles the race condition where a concurrent request
        # creates the same SKU between our check and our insert.
        return jsonify({
            'error': 'SKU already exists (concurrent write conflict)'
        }), 409

    except Exception:
        db.session.rollback()
        # FIX 8: Log the exception to your observability platform
        # (e.g., Sentry, Datadog). Never expose raw exception details
        # in an API response.
        return jsonify({'error': 'Internal server error'}), 500

    # FIX 7: Return 201 Created (not 200 OK) for successful resource creation
    return jsonify({
        'message': 'Product created successfully',
        'product_id': product.id,
        'sku': product.sku
    }), 201
```

Part 2 — Database Schema Design

2.1 Assumptions & Design Philosophy

Key Assumption: Multi-tenancy is enforced at the company level. Each company owns its own warehouses, products, and supplier relationships. There is no cross-company product sharing.

- Monetary values (price, cost) use NUMERIC(12, 2) — never FLOAT.
- All tables use surrogate integer primary keys for join performance. Business keys (SKU, email) are given explicit UNIQUE constraints.
- Soft deletes (deleted_at TIMESTAMPTZ) are used on products rather than hard deletes, to preserve audit history.
- created_at / updated_at are managed by the ORM or DB triggers, not application code.
- Inventory history uses an append-only ledger pattern (inventory_transactions). The inventory table holds the current stock level; the ledger holds the full history of changes.

2.2 Full SQL Schema (PostgreSQL)

```
-- ═══════════════════════════════════════════════════════════════
-- STOCKFLOW DATABASE SCHEMA — PostgreSQL 14+
-- Tables are ordered so that each foreign key reference points
-- to a table already defined above it in this file.
-- ═══════════════════════════════════════════════════════════════

-- ───────────────────────────────────────────────────────────────
-- COMPANIES
-- Root tenant entity. All other tables belong to a company.
-- ───────────────────────────────────────────────────────────────
CREATE TABLE companies (
    id          SERIAL PRIMARY KEY,
    name        VARCHAR(255) NOT NULL,
    slug        VARCHAR(100) UNIQUE NOT NULL,  -- URL-safe identifier
    email       VARCHAR(255) UNIQUE NOT NULL,
    plan_tier   VARCHAR(50)  NOT NULL DEFAULT 'starter',
    created_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    deleted_at  TIMESTAMPTZ                      -- soft delete
);

-- ───────────────────────────────────────────────────────────────
-- USERS
-- Defined early so inventory_transactions can reference it.
-- Users are scoped to a single company (no cross-tenant accounts).
-- ───────────────────────────────────────────────────────────────
CREATE TABLE users (
    id          SERIAL PRIMARY KEY,
    company_id  INT         NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
    email       VARCHAR(255) UNIQUE NOT NULL,
    role        VARCHAR(50)  NOT NULL DEFAULT 'member',  --
'owner','admin','member'
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_users_company ON users(company_id);

-- ───────────────────────────────────────────────────────────────
-- WAREHOUSES
```

```
-- A company can have many warehouses.
-- ────────────────────────────────────────────────────────────────
CREATE TABLE warehouses (
    id            SERIAL PRIMARY KEY,
    company_id    INT          NOT NULL REFERENCES companies(id) ON DELETE
CASCADE,
    name          VARCHAR(255) NOT NULL,
    address_line1 VARCHAR(255),
    city          VARCHAR(100),
    country       CHAR(2),                -- ISO 3166-1 alpha-2
    timezone      VARCHAR(64),             -- e.g. 'America/New_York'
    is_active     BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_warehouses_company ON warehouses(company_id);

-- ────────────────────────────────────────────────────────────────
-- PRODUCT CATEGORIES
-- Carries the low_stock_threshold used by the alert query.
-- A per-product override (reorder_point) in the inventory table
-- takes precedence when set.
-- ────────────────────────────────────────────────────────────────
CREATE TABLE product_categories (
    id                  SERIAL PRIMARY KEY,
    company_id          INT          NOT NULL REFERENCES companies(id) ON DELETE
CASCADE,
    name                VARCHAR(100) NOT NULL,
    low_stock_threshold INT          NOT NULL DEFAULT 10,
    UNIQUE (company_id, name)
);

-- ────────────────────────────────────────────────────────────────
-- PRODUCTS
-- Master product catalogue. Not tied to any specific warehouse.
-- Warehouse presence is recorded in the inventory table.
-- SKUs are unique within a company, not globally.
-- ────────────────────────────────────────────────────────────────
CREATE TABLE products (
    id            SERIAL          PRIMARY KEY,
    company_id    INT             NOT NULL REFERENCES companies(id) ON DELETE
CASCADE,
    category_id   INT             REFERENCES product_categories(id) ON DELETE SET
NULL,
    name          VARCHAR(255)    NOT NULL,
    sku           VARCHAR(100)    NOT NULL,
    description   TEXT,
    price         NUMERIC(12, 2)  NOT NULL CHECK (price >= 0),
    cost_price    NUMERIC(12, 2)  CHECK (cost_price >= 0),
    is_bundle     BOOLEAN         NOT NULL DEFAULT FALSE,
    is_active     BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at    TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    deleted_at    TIMESTAMPTZ,
    UNIQUE (company_id, sku)      -- SKU unique within a company
);
CREATE INDEX idx_products_company  ON products(company_id);
CREATE INDEX idx_products_category ON products(category_id);

-- ────────────────────────────────────────────────────────────────
-- BUNDLE COMPONENTS
```

```
-- Self-referencing M2M on products. Records which child products
-- make up a bundle and in what quantity.
-- ────────────────────────────────────────────────────────────────
CREATE TABLE bundle_components (
    bundle_product_id    INT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    component_product_id INT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
    quantity             INT NOT NULL DEFAULT 1 CHECK (quantity > 0),
    PRIMARY KEY (bundle_product_id, component_product_id),
    CHECK (bundle_product_id <> component_product_id)  -- prevent self-reference
);

-- ────────────────────────────────────────────────────────────────
-- SUPPLIERS
-- Global entities. A supplier can work with multiple companies.
-- The company_suppliers table records which suppliers a company uses.
-- ────────────────────────────────────────────────────────────────
CREATE TABLE suppliers (
    id            SERIAL PRIMARY KEY,
    name          VARCHAR(255) NOT NULL,
    contact_email VARCHAR(255),
    contact_phone VARCHAR(50),
    website       VARCHAR(255),
    address_line1 VARCHAR(255),
    country       CHAR(2),
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Which suppliers a company has a relationship with
CREATE TABLE company_suppliers (
    company_id     INT         NOT NULL REFERENCES companies(id)  ON DELETE
CASCADE,
    supplier_id    INT         NOT NULL REFERENCES suppliers(id)  ON DELETE
CASCADE,
    account_number VARCHAR(100),
    PRIMARY KEY (company_id, supplier_id)
);

-- Which supplier provides which product for a given company.
-- is_primary flags the preferred supplier for reorder alerts.
CREATE TABLE product_suppliers (
    id             SERIAL         PRIMARY KEY,
    product_id     INT            NOT NULL REFERENCES products(id)  ON DELETE
CASCADE,
    supplier_id    INT            NOT NULL REFERENCES suppliers(id) ON DELETE
CASCADE,
    supplier_sku   VARCHAR(100),              -- Supplier's own part number
    unit_cost      NUMERIC(12, 2),
    lead_time_days INT,                       -- Days from order to delivery
    is_primary     BOOLEAN        NOT NULL DEFAULT FALSE,
    UNIQUE (product_id, supplier_id)
);
CREATE INDEX idx_product_suppliers_product ON product_suppliers(product_id);

-- ────────────────────────────────────────────────────────────────
-- INVENTORY
-- Current stock level of a product at a specific warehouse.
-- One row per (product, warehouse) pair.
-- reorder_point overrides the category-level threshold when set.
-- ────────────────────────────────────────────────────────────────
CREATE TABLE inventory (
    id            SERIAL PRIMARY KEY,
```

```
    product_id    INT     NOT NULL REFERENCES products(id)   ON DELETE CASCADE,
    warehouse_id  INT     NOT NULL REFERENCES warehouses(id) ON DELETE CASCADE,
    quantity      INT     NOT NULL DEFAULT 0 CHECK (quantity >= 0),
    reorder_point INT,    -- Per-product-warehouse threshold override
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (product_id, warehouse_id)
);
CREATE INDEX idx_inventory_warehouse ON inventory(warehouse_id);
CREATE INDEX idx_inventory_product   ON inventory(product_id);

-- ────────────────────────────────────────────────────────────────
-- INVENTORY TRANSACTIONS (Append-Only Ledger)
-- Never updated or deleted. Each row records a single stock
-- movement. Positive delta = stock in; negative delta = stock out.
-- The inventory table reflects the running sum of all deltas.
-- ────────────────────────────────────────────────────────────────
CREATE TABLE inventory_transactions (
    id               BIGSERIAL PRIMARY KEY,
    product_id       INT         NOT NULL REFERENCES products(id),
    warehouse_id     INT         NOT NULL REFERENCES warehouses(id),
    delta            INT         NOT NULL,
    transaction_type VARCHAR(50) NOT NULL,
       -- 'sale' | 'purchase' | 'adjustment' | 'transfer_in' | 'transfer_out'
    reference_id     INT,          -- FK to orders, purchase_orders, etc.
    reference_type   VARCHAR(50),  -- 'order' | 'purchase_order' | 'manual'
    performed_by     INT REFERENCES users(id) ON DELETE SET NULL,
    notes            TEXT,
    created_at       TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- Partial index: accelerates the recent-sales subquery in the
-- low-stock alert endpoint.
CREATE INDEX idx_inv_txn_sales
    ON inventory_transactions(product_id, warehouse_id, created_at DESC)
    WHERE transaction_type = 'sale';
```

2.3 Design Decisions & Justifications

DDL Table Order

Tables are defined in dependency order so that every REFERENCES clause points to an already-created table. In particular, users is defined before inventory_transactions so that the performed_by foreign key is valid. In a production migration tool (e.g., Alembic), this ordering is enforced automatically, but it matters for raw DDL execution.

Inventory: Current State + Append-Only Ledger

The inventory table stores the current quantity for fast reads. The inventory_transactions table stores every individual movement as an immutable row. These two are kept in sync by application logic (or a DB trigger): every write to inventory_transactions also updates inventory.quantity by the same delta. This separation gives O(1) stock lookups while preserving a full audit trail.

Bundle Support via Self-Referencing M2M

bundle_components is a self-referencing many-to-many table on products. The is_bundle flag on the product row enables fast filtering (e.g., 'exclude bundles from alerts') without a join. A CHECK constraint prevents a product from listing itself as a component; preventing deeper cycles requires application-level validation.

SKU Scope: Per Company, Not Global

SKUs are unique within a company via UNIQUE(company_id, sku). Different companies may legitimately reuse the same SKU values, which is standard in B2B SaaS. This design is consistent across the schema, the fixed API code in Part 1, and the assumptions table at the end of this document.

Threshold Resolution Order

The low-stock threshold is resolved at query time using: COALESCE(inventory.reorder_point, product_categories.low_stock_threshold, 10). This gives three levels of control: a per-product-warehouse override, a category-level default, and a system-wide fallback.

2.4 Questions for the Product Team

- Should inventory quantity be allowed to go negative to support backorders, or must it always be >= 0?
- Can a product have multiple active suppliers at the same time, or is exactly one primary supplier required at all times?
- For bundle products, is stock tracked on the bundle itself, or is it always derived from its component quantities?
- What is the 'recent sales activity' window for low-stock alerts — 7 days, 30 days? Should this be configurable per company or per product type?
- Should warehouse-to-warehouse stock transfers be recorded as two linked transactions (transfer_out + transfer_in) or as a single transfer event?
- Is soft delete required for all entities, or only for products? Should deleted products still appear in historical reports?
- Are user accounts strictly company-scoped, or can a consultant-type user belong to multiple companies?

Part 3 — Low-Stock Alert API Implementation

3.1 Assumptions

Key Assumption: "Recent sales activity" is defined as at least one sale transaction within the past 30 days (configurable via RECENT_SALES_DAYS). Sales velocity is the average daily units sold over this same window. days_until_stockout = FLOOR(current_stock / avg_daily_sales). If a product has

> no sales in the window it is excluded entirely; if it has sales but zero velocity (edge case), days_until_stockout is returned as null.

- Bundle products are excluded from alerts — their effective stock is derived from components and cannot be independently restocked.
- The endpoint is paginated (limit / offset) to safely handle companies with large product catalogues.
- The company_id in the URL must match the authenticated user's company. A mismatch returns 403.
- avg_daily_sales is included in the response as a useful diagnostic field, even though it is not in the original spec. It can be removed if not needed.

3.2 Full Implementation

```
# ────────────────────────────────────────────────────────────────────
# StockFlow — Low-Stock Alert Endpoint
# GET /api/companies/<company_id>/alerts/low-stock
# ────────────────────────────────────────────────────────────────────
from flask import Blueprint, request, jsonify, g
from sqlalchemy import text
from datetime import datetime, timezone
from models import db
from auth import login_required

alerts_bp = Blueprint('alerts', __name__)

RECENT_SALES_DAYS = 30   # Defines 'recent sales activity'
DEFAULT_LIMIT     = 50
MAX_LIMIT         = 200  # Safety cap to prevent oversized responses


@alerts_bp.route('/api/companies/<int:company_id>/alerts/low-stock',
                 methods=['GET'])
@login_required
def get_low_stock_alerts(company_id):
    """
    Return low-stock alerts for all warehouses belonging to a company.

    Threshold resolution order:
      1. inventory.reorder_point   (per product-warehouse override)
      2. product_categories.low_stock_threshold  (per category)
      3. System default: 10

    Only products with at least one sale in the last RECENT_SALES_DAYS
    are included. Bundle products are excluded.

    Results are sorted by urgency: lowest days_until_stockout first,
    with null values (no measurable velocity) last.
    """

    # ── Authorization ──────────────────────────────────────────────
    if g.current_user.company_id != company_id:
```

```
        return jsonify({'error': 'Access denied'}), 403

    # ── Query Parameters ──────────────────────────────────────────────
    try:
        limit = min(int(request.args.get('limit', DEFAULT_LIMIT)), MAX_LIMIT)
        offset = int(request.args.get('offset', 0))
        warehouse_id_filter = request.args.get('warehouse_id', type=int)
    except (TypeError, ValueError):
        return jsonify({'error': 'Invalid pagination parameters'}), 400

    # ── SQL Query ─────────────────────────────────────────────────────
    #
    # The recent_sales CTE aggregates sale transactions per
    # (product, warehouse) pair within the lookback window.
    #
    # The main query then joins this CTE with inventory, products,
    # warehouses, categories, and the primary supplier.
    #
    # Key decisions:
    #   - INNER JOIN on recent_sales enforces the 'recent activity' rule.
    #   - COALESCE resolves the threshold at three levels.
    #   - Division by zero is avoided with a CASE expression.
    #   - LEFT JOIN on suppliers ensures products without a primary
    #     supplier still appear in the results.
    #
    query = text("""
        WITH recent_sales AS (
            SELECT
                it.product_id,
                it.warehouse_id,
                SUM(ABS(it.delta))::FLOAT / :sales_days  AS avg_daily_sales
            FROM  inventory_transactions it
            JOIN  warehouses w ON w.id = it.warehouse_id
            WHERE w.company_id        = :company_id
              AND it.transaction_type = 'sale'
              AND it.created_at       >= NOW() - INTERVAL '1 day' * :sales_days
            GROUP BY it.product_id, it.warehouse_id
        )
        SELECT
            p.id                              AS product_id,
            p.name                            AS product_name,
            p.sku,
            w.id                              AS warehouse_id,
            w.name                            AS warehouse_name,
            inv.quantity                      AS current_stock,
            COALESCE(
                inv.reorder_point,
                pc.low_stock_threshold,
                10
            )                                 AS threshold,
            rs.avg_daily_sales,
            CASE
                WHEN rs.avg_daily_sales > 0
                THEN FLOOR(inv.quantity / rs.avg_daily_sales)::INT
                ELSE NULL
            END                               AS days_until_stockout,
            s.id                              AS supplier_id,
            s.name                            AS supplier_name,
            s.contact_email                   AS supplier_email
        FROM       inventory          inv
        JOIN       products           p   ON p.id  = inv.product_id
        JOIN       warehouses         w   ON w.id  = inv.warehouse_id
```

```
        -- INNER JOIN: only products with recent sales activity
        JOIN       recent_sales        rs  ON rs.product_id   = inv.product_id
                                          AND rs.warehouse_id = inv.warehouse_id
        LEFT JOIN  product_categories  pc  ON pc.id = p.category_id
        -- LEFT JOIN: products with no primary supplier still appear
        LEFT JOIN  product_suppliers   ps  ON ps.product_id = p.id
                                          AND ps.is_primary  = TRUE
        LEFT JOIN  suppliers           s   ON s.id = ps.supplier_id
        WHERE  w.company_id  = :company_id
          AND  p.is_active   = TRUE
          AND  p.deleted_at  IS NULL
          AND  p.is_bundle   = FALSE
          -- Stock is below the resolved threshold
          AND  inv.quantity  < COALESCE(
                                 inv.reorder_point,
                                 pc.low_stock_threshold,
                                 10
                             )
          AND  (:warehouse_id IS NULL OR w.id = :warehouse_id)
        ORDER BY
            days_until_stockout ASC NULLS LAST,
            current_stock       ASC
        LIMIT  :lim
        OFFSET :off
    """)

    count_query = text("""
        WITH recent_sales AS (
            SELECT it.product_id, it.warehouse_id
            FROM   inventory_transactions it
            JOIN   warehouses w ON w.id = it.warehouse_id
            WHERE  w.company_id        = :company_id
              AND  it.transaction_type = 'sale'
              AND  it.created_at       >= NOW() - INTERVAL '1 day' * :sales_days
            GROUP BY it.product_id, it.warehouse_id
        )
        SELECT COUNT(*) AS total
        FROM      inventory         inv
        JOIN      products          p   ON p.id = inv.product_id
        JOIN      warehouses        w   ON w.id = inv.warehouse_id
        JOIN      recent_sales       rs  ON rs.product_id   = inv.product_id
                                        AND rs.warehouse_id = inv.warehouse_id
        LEFT JOIN product_categories pc  ON pc.id = p.category_id
        WHERE  w.company_id = :company_id
          AND  p.is_active  = TRUE
          AND  p.deleted_at IS NULL
          AND  p.is_bundle  = FALSE
          AND  inv.quantity < COALESCE(
                                inv.reorder_point,
                                pc.low_stock_threshold,
                                10
                            )
          AND  (:warehouse_id IS NULL OR w.id = :warehouse_id)
    """)

    params = {
        'company_id':  company_id,
        'sales_days':  RECENT_SALES_DAYS,
        'warehouse_id': warehouse_id_filter,
        'lim':         limit,
        'off':         offset
    }
```

```
    count_params = {k: v for k, v in params.items()
                    if k not in ('lim', 'off')}

    # ── Execute ──────────────────────────────────────────────────────
    try:
        rows        = db.session.execute(query, params).fetchall()
        total_count = db.session.execute(count_query, count_params).scalar()
    except Exception:
        # Log exception to observability platform before returning
        return jsonify({'error': 'Internal server error'}), 500

    # ── Build Response ───────────────────────────────────────────────
    alerts = []
    for row in rows:
        alert = {
            'product_id':         row.product_id,
            'product_name':       row.product_name,
            'sku':                row.sku,
            'warehouse_id':       row.warehouse_id,
            'warehouse_name':     row.warehouse_name,
            'current_stock':      row.current_stock,
            'threshold':          row.threshold,
            'avg_daily_sales':    round(row.avg_daily_sales, 2)
                                  if row.avg_daily_sales else 0,
            'days_until_stockout': row.days_until_stockout,  # None = no velocity
            'supplier':           None
        }
        if row.supplier_id:
            alert['supplier'] = {
                'id':            row.supplier_id,
                'name':          row.supplier_name,
                'contact_email': row.supplier_email
            }
        alerts.append(alert)

    return jsonify({
        'alerts':       alerts,
        'total_alerts': total_count,
        'page': {
            'limit':    limit,
            'offset':   offset,
            'has_more': (offset + limit) < total_count
        },
        'metadata': {
            'sales_window_days': RECENT_SALES_DAYS,
            'generated_at':      datetime.now(timezone.utc).isoformat()
        }
    }), 200
```

3.3 Edge Cases Handled

- Company ID mismatch — 403 returned before any query is executed.
- Products with no sales in the window — excluded via INNER JOIN on the recent_sales CTE. They do not appear in results even if below threshold.
- Products with no primary supplier — supplier field is null in the response; no crash, because the join is a LEFT JOIN.

- Zero or undefined sales velocity — days_until_stockout is null (CASE expression avoids division by zero).
- Bundle products — explicitly excluded by the WHERE clause (p.is_bundle = FALSE).
- Invalid pagination parameters — caught before any DB call and returned as 400.
- Database errors — caught in a blanket except; 500 returned with no internal detail exposed.
- Pagination overflow — has_more flag lets clients detect the end of results without issuing an extra empty-page request.

3.4 Production Enhancements (Beyond Scope)

- Caching: This query involves several joins and is a strong candidate for short-lived caching in Redis (e.g., 5 minutes, keyed by company_id). Cache should be invalidated on any inventory_transactions insert for that company.
- Materialised view / background job: For very large catalogues, a Celery task could pre-compute alerts into a dedicated table on a scheduled basis, reducing endpoint latency to a simple SELECT.
- Push notifications / webhooks: Rather than polling this endpoint, companies could subscribe to threshold-crossing events and receive a webhook or email when a product first drops below its threshold.
- Rate limiting: Per-company rate limits should be applied to prevent abuse, particularly since the query touches multiple large tables.

Summary of Assumptions

Area	Assumption Made
SKU uniqueness	Unique per company via UNIQUE(company_id, sku) — not globally across the platform. Consistent across schema, API code, and this table.
Authentication	@login_required decorator sets g.current_user; JWT-based auth assumed.
Price storage	NUMERIC(12, 2) in the DB; Python Decimal in application code — never float.
Recent sales window	30 days, configurable via the RECENT_SALES_DAYS constant.
Bundle stock	Not independently tracked. Bundles are excluded from low-stock alerts.
days_until_stockout	FLOOR(current_stock / avg_daily_sales). Returns null when avg_daily_sales is zero.
Multi-tenancy	company_id is enforced on every query. No cross-tenant data access is possible.

Soft deletes	Products use a deleted_at column. Hard deletes are not permitted to preserve audit history.
Users	Strictly company-scoped (one company per user account).